

Decentralized Multi-Drone Swarm Coordination: A 100-Agent Simulation Framework

1. Abstract

This project presents the design and implementation of a **decentralized coordination framework** for simulating 100 autonomous drones in a shared 2D environment. The system eliminates centralized control bottlenecks by enabling each drone to make decisions based solely on **local sensing, neighbor communication, and lightweight consensus protocols**. Core algorithmic contributions span **bio-inspired flocking (Reynolds model), reactive and predictive collision avoidance, proximity-based decentralized task allocation, and dynamic fault-tolerant reconfiguration**. Simulation operates on discrete time-steps with simplified kinematics, spatial partitioning for scalable neighbor queries, and real-time visual rendering. The framework targets emergent collective behavior—demonstrating that complex, coordinated missions (area coverage, target tracking, formation traversal) arise from simple local interaction rules applied at scale.

2. Introduction & Motivation

Modern applications of unmanned aerial vehicle (UAV) swarms—disaster response, precision agriculture, environmental monitoring, infrastructure inspection, and defense—demand systems that can coordinate **dozens to hundreds of agents** without relying on a single point of failure. Centralized architectures suffer from communication bottlenecks, latency, and catastrophic failure if the coordinator is compromised.

Decentralized swarm coordination addresses these limitations by distributing decision-making to each agent. Each drone operates with only local information—its own state and that of nearby neighbors—and employs lightweight algorithms to achieve globally coherent behavior. This paradigm draws heavily from biological systems (bird flocks, fish schools, ant colonies) and has seen rapid research acceleration in 2024–2026.

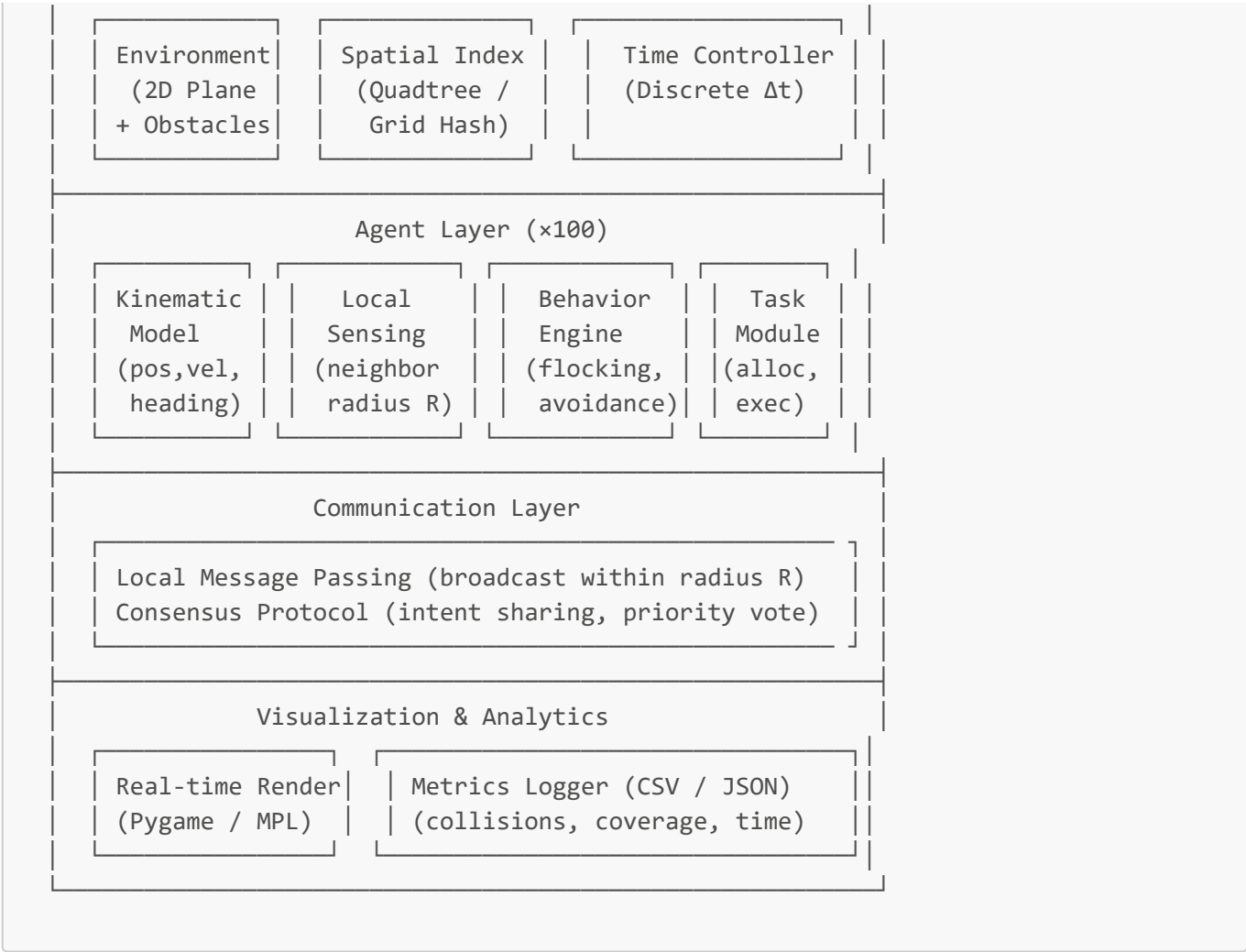
Why 100 Drones?

Scaling to 100 agents is a meaningful benchmark:

- It is large enough to expose **non-linear coordination complexity** (communication overhead, collision density, task contention).
- It requires **spatial indexing** (quadtree/grid partitioning) to maintain real-time performance.
- It serves as a proving ground for algorithms before deployment on even larger swarms (1,000+).
- Most published simulations validate on 16–36 agents; demonstrating robust behavior at 100 is a contribution in itself.

3. System Architecture

Simulation Engine



4. Technical Approach

4.1 Environment Modeling

Parameter	Specification
Space	2D continuous plane (default 1000 × 1000 units)
Boundaries	Configurable: wrap-around (toroidal) or hard boundary with repulsion
Obstacles	Static polygons + optional dynamic obstacles with predefined trajectories
Time Step	Discrete Δt = 0.02–0.05 s (20–50 ms per tick)

4.2 Drone Kinematic Model

Each drone i maintains state vector $\mathbf{s}_i = (\mathbf{x}, \mathbf{y}, \mathbf{v}_x, \mathbf{v}_y, \theta)$ and updates per tick:

```
x_{t+1} = x_t + v_{x,t} \cdot \Delta t
y_{t+1} = y_t + v_{y,t} \cdot \Delta t
\theta_{t+1} = \text{atan2}(v_{y,t+1}, v_{x,t+1})
```

Velocity is bounded: $|v| \leq v_{max}$. Acceleration is bounded: $|a| \leq a_{max}$ to prevent unrealistic instantaneous direction changes.

4.3 Local Sensing & Neighbor Discovery

- **Interaction Radius (R):** Each drone perceives neighbors within radius R (default R = 50 units).
- **Spatial Partitioning:** A **grid-based hash** or **quadtree** reduces neighbor search from $O(n^2)$ to approximately $O(n \log n)$ or $O(n)$.
- **Sensed Data:** Relative position, relative velocity, heading, and task status of each neighbor.

4.4 Behavior Engine: Reynolds Flocking Model

Three steering forces computed per drone per tick, weighted and summed:

Rule	Description	Force Formula
Separation	Steer away from neighbors closer than safety distance d_s	$F_{sep} = -\sum (pos_j - pos_i) / pos_j - pos_i ^2$ for $ \Delta < d_s$
Alignment	Match average heading of neighbors	$F_{align} = avg(v_j) - v_i$
Cohesion	Steer toward centroid of neighbor positions	$F_{coh} = centroid(pos_j) - pos_i$
Obstacle Avoidance	Repulsive force from nearby obstacles	$F_{obs} = -\sum (obs_k - pos_i) / obs_k - pos_i ^2$

Combined steering: $a_i = w_s \cdot F_{sep} + w_a \cdot F_{align} + w_c \cdot F_{coh} + w_o \cdot F_{obs} + w_t \cdot F_{task}$

Weights (w_s, w_a, w_c, w_o, w_t) are tunable parameters governing emergent behavior profiles.

4.5 Collision Avoidance

Two-layer approach:

1. **Reactive Layer:** If distance to any neighbor $< d_{safety}$, apply strong repulsive force immediately (override other steering).
2. **Predictive Layer:** Project positions forward by k time-steps; if projected collision detected, apply preemptive velocity adjustment.

4.6 Decentralized Task Allocation

Tasks (e.g., waypoints to visit, zones to cover, targets to track) are broadcast or pre-seeded in the environment. Drones self-assign tasks using:

- **Proximity-based greedy:** Assign to nearest unoccupied task.
- **Auction protocol:** Drones within communication range bid on tasks based on estimated cost (distance + energy); lowest-cost drone wins.
- **Dynamic reallocation:** If a drone fails or a new task appears, neighbors detect the change and re-bid within 1–2 communication rounds.

4.7 Consensus & Communication Protocol

- **Local broadcast:** Each drone broadcasts its intent (current task, heading, status) to all neighbors within radius R.
- **Priority voting:** When conflicting intents are detected (e.g., two drones claiming same task), a lightweight **max-ID** or **min-cost** consensus resolves the conflict in $O(\text{diameter})$ rounds.
- **Message format:** {drone_id, position, velocity, task_id, task_status, timestamp}

4.8 Fault Tolerance & Dynamic Reconfiguration

- **Heartbeat mechanism:** If a drone's broadcast is not received for T_{timeout} ticks, neighbors mark it as failed.
- **Task recovery:** Failed drone's task is released back to the available pool; nearest capable neighbor re-claims it.
- **Formation healing:** Flocking rules naturally fill gaps left by failed drones, maintaining group cohesion.

5. Implementation Milestones

Milestone 1: Foundation (Weeks 1–3)

Deliverable	Details
2D simulation environment	Continuous plane with static obstacles, configurable boundaries
Drone agent class	State vector, kinematic update, parameter configuration
Spatial partitioning	Grid hash or quadtree for $O(n)$ neighbor queries
Local sensing	Neighbor detection within radius R with relative state computation
Basic visualization	Real-time rendering of drone positions, headings, and neighbor links
Reactive collision avoidance	Safety-distance repulsion between drones and obstacles

Validation: 100 drones spawn randomly and move without collisions for 1,000+ ticks.

Milestone 2: Coordination Algorithms (Weeks 4–6)

Deliverable	Details
Reynolds flocking	Separation + Alignment + Cohesion with tunable weights
Formation control	Predefined shapes (V-formation, grid, circle) via target-position injection
Decentralized task allocation	Proximity-based + auction-based assignment with local messaging
Consensus protocol	Max-ID conflict resolution for task contention
Enhanced collision avoidance	Predictive layer integrated with steering forces

Validation: 100 drones form cohesive flocks, dynamically split/merge, and allocate tasks without central coordinator.

Milestone 3: Mission Execution & Evaluation (Weeks 7–9)

Deliverable	Details
Mission scenarios	Area coverage, target tracking, formation traversal through obstacle field
Fault injection	Random drone removal (10%, 25%) mid-mission; observe recovery
Dynamic obstacles	Moving obstacles requiring real-time path adaptation
Performance metrics suite	Completion time, collision count, coverage %, formation error, message count
Comparative analysis	Centralized vs. decentralized; impact of swarm size (25, 50, 75, 100)
Final visualization	Polished demo with trails, heatmaps, and metric overlays

Validation: Quantitative analysis showing decentralized approach maintains >90% task completion even with 25% drone attrition.

6. Evaluation Metrics

Metric	Description	Target
Mission Completion Time	Ticks to complete all assigned tasks	Minimize
Collision Count	Total inter-drone and drone-obstacle collisions	0 (ideal)
Area Coverage %	Fraction of target area covered within time limit	>95%
Formation Error	Average deviation from target formation positions	<5% of inter-agent spacing
Robustness Index	Coverage maintained after N% drone failures	>90% at 25% failure
Message Overhead	Total messages sent per tick per drone	<10 messages/tick/drone
Scalability	Performance degradation as drone count increases	Sub-linear

7. Technology Stack

Component	Tool
Language	Python 3.10+
Visualization	Pygame (real-time) + Matplotlib (analysis plots)
Spatial indexing	Custom quadtree or <code>scipy.spatial.KDTree</code>
Data logging	CSV / JSON with pandas for post-processing
Reproducibility	Fixed random seeds, configurable YAML parameters
Version control	Git with milestone-based branching

8. Project Timeline

```
Week 1-2: Environment + drone model + spatial indexing
Week 3:   Collision avoidance + basic visualization
Week 4:   Reynolds flocking + formation control
Week 5:   Task allocation + consensus protocol
Week 6:   Integration testing + parameter tuning
Week 7:   Mission scenarios + fault injection
Week 8:   Performance evaluation + comparative analysis
Week 9:   Final demo + documentation + report
```

9. References & Related Work

1. Reynolds, C. W. (1987). *Flocks, Herds, and Schools: A Distributed Behavioral Model*. ACM SIGGRAPH.
2. Olfati-Saber, R. (2006). *Flocking for Multi-Agent Dynamic Systems*. IEEE TAC.
3. Vásárhelyi, G. et al. (2018). *Optimized Flocking of Autonomous Drones in Confined Environments*. Science Robotics.
4. Tordesillas, J. & How, J. P. (2024). *Robust MADER: Decentralized and Asynchronous Multi-Agent Trajectory Planner Robust to Communication Delays*. MIT, IEEE RA-L.
5. GPU-accelerated batch-parallel multirotor simulators for large-scale swarm RL (2024–2025).
6. Decentralized swarm control with behavior-based control + fuzzy logic + virtual leader mechanisms (Sept. 2025).
7. Center-sub-critics RL for multi-UAV cooperative search-and-track (Feb. 2026).

Document Version 2.0 — Prepared for PDC Course Project, Spring 2026